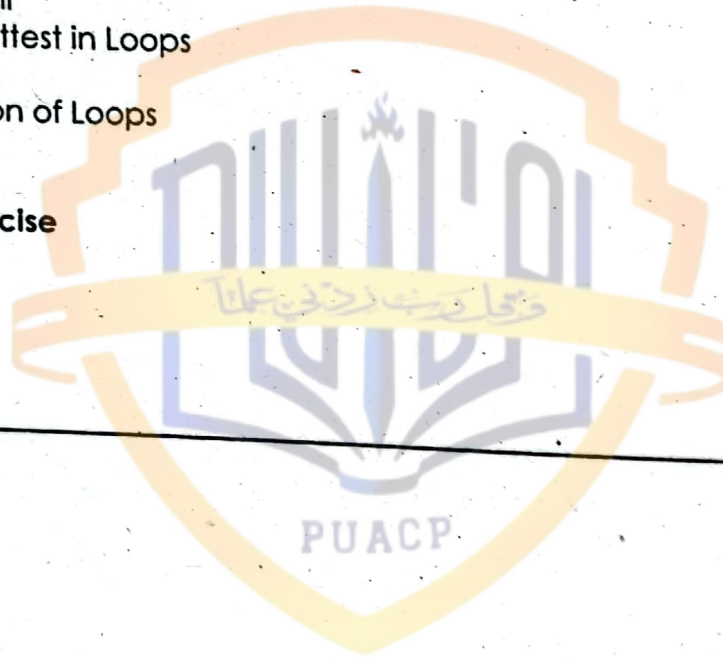


Chapter Overview**6.1 Looping Structure****6.1.1 Counter-Controlled Loops****6.1.2 Sentinel-Controlled Loops****6.2 While...Wend Loop****6.3 Do...While...Loop****6.4 Do...Until...Loop****6.5 Do...Loop...While****6.6 Do...Loop...Until****6.7 Pretest and Posttest in Loops****6.8 For...Next Loop****6.9 Early Termination of Loops****6.10 Nested Loops****Programming Exercise****Short Questions****Multiple Choices****Fill in the Blanks****True/False**

6.1 Looping Structure

A type of control structure that repeats a statement or set of statements is known as looping structure. It is also known as iterative or repetitive structure. In sequential structure, all statements are executed once. On the other hand, conditional structure may execute or skip a statement on the basis of given condition. In some situations, it is required to repeat a statement or number of statements for a specified number of times. Looping structures are used for this purpose. There are different types of loops available in Visual Basic. Loops are basically used for two purposes:

- To execute a statement or number of statements for a specified number of times. For example, you may want to display a message for 100 times etc.
- To use a sequence of values. For example, to display first 50 natural numbers etc.

6.1.1 Counter-Controlled Loops

This type of loop depends on the value of a variable known as **counter variable**. The value of counter variable is incremented or decremented each time the body of the loop is executed. This loop terminates when the value of counter variable reaches a particular value. The number of iterations of counter-controlled loop is known in advance. The iterations of this loop depends on the following:

- Initial value of the counter variable
- Terminating condition
- Increment or decrement value

6.1.2 Sentinel-Controlled Loops

This type of loop depends on special value known as **sentinel value**. Sentinel value indicates that the loop should **continue or terminate**. These types of loops are also known as **conditional loops**. For example, a loop may execute while the value of a variable is not -1. Here -1 is the sentinel value used to terminate loop.

The number of iterations of **sentinel-controlled loop** is **unknown**. It depends on the input from the user. Suppose the **sentinel value to terminate a loop** is -1. If the user enters -1 in first iteration, the loop will execute **only once**. But if the user enters -1 after entering many other values, the number of iterations will vary accordingly.

6.2 While...Wend Loop

While...Wend is the simplest loop of Visual BASIC language. It executes one or more statements while the given condition remains **true**.

Syntax

The syntax of **While...Wend** loop is as follows:

```
While condition
    statement(s)
```

```
Wend
```

While condition

It indicates the start of loop.

It is given as relational expression. The loop executes as long as the condition remains true.

statements(s)

It indicates the statement or set of statements to be repeated by the loop.

Wend

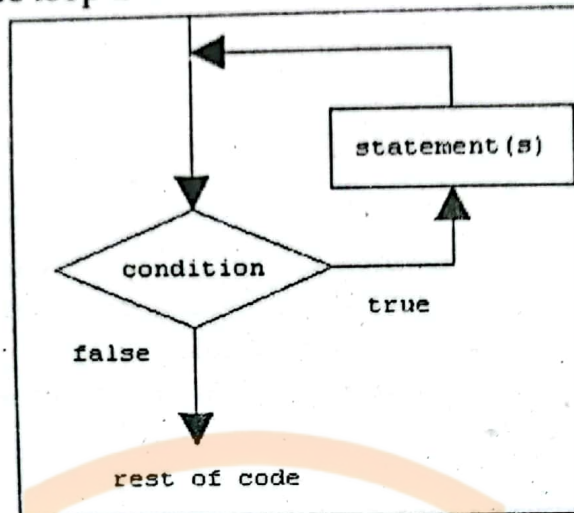
It indicates the end of loop. The statements written between **While** and **Wend** are repeated by the loop.

Working of While...Wend Loop

First of all, the condition is evaluated. If it is true, the control enters the body of the loop and executes all statements. After executing the statements, it again moves to the start of the loop and evaluates the condition again. This process continues as long as the condition remains true. When the condition becomes false, the loop is terminated.

Flowchart

The flowchart of While loop is as follows:



Program 6.1

Write a program that prints "Hello world!" five times on form using While loop.

1. Start a new Standard EXE project.
2. Place a command button on the form as follows:

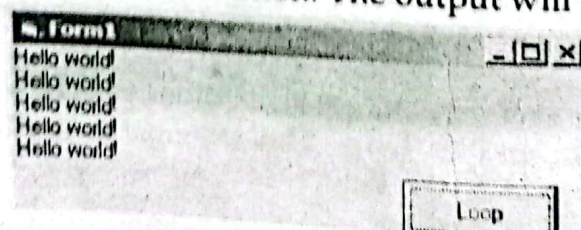


3. Double click the button and add the following code in its Click event:

```

Private Sub Command1_Click()
    Dim n As Integer
    n = 1
    While n <= 5
        Print "Hello world!"
        n = n + 1
    Wend
End Sub
  
```

4. Run the project and click the button. The output will be as follows:



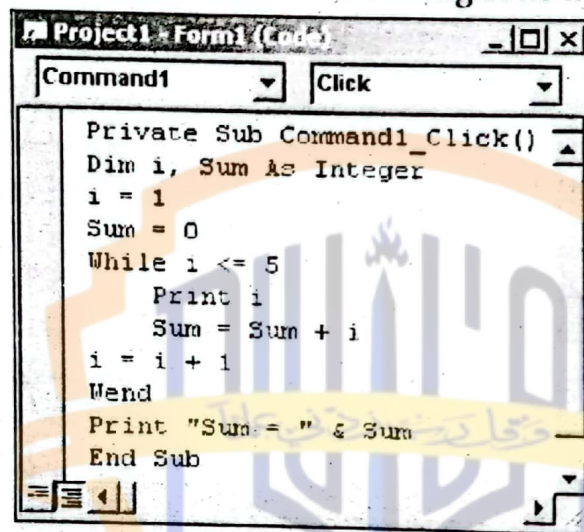
How It Works?

The above example is a counter-controlled loop. It uses a counter variable *n* to control the iterations of the loop. The variable *n* is initialized to 1. When the condition is evaluated first time, the value of *n* is 1. The condition is true, so the control enters the body of the loop that contains two statements. The first statement prints "Hello World!" on the form and the second statement increments the value of *n* by 1, making it 2. Then the control moves back to the condition. This process continues for five times. When the value of *n* becomes 6, the condition becomes false and the loop terminates.

Program 6.2

Write a program that displays first five numbers and their sum using While loop.

1. Start a new Standard EXE project.
2. Place a button on the form.
3. Double click the button and add the following code in its Click event.



```

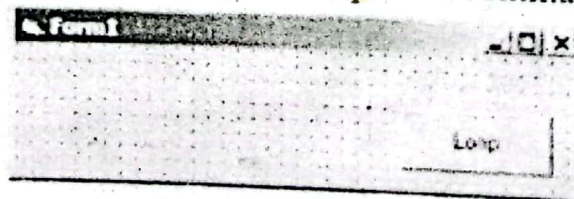
Private Sub Command1_Click()
    Dim i, Sum As Integer
    i = 1
    Sum = 0
    While i <= 5
        Print i
        Sum = Sum + i
        i = i + 1
    Wend
    Print "Sum = " & Sum
End Sub
  
```

4. Run the project and test the working.

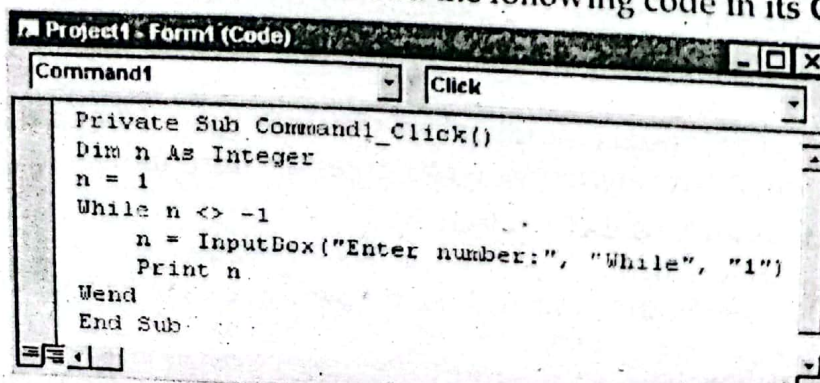
Program 6.3

Write a program that inputs a number and displays it on form until the user input -1.

1. Start a new Standard EXE project and place a command button on the form:



2. Double click on the button and add the following code in its Click event:



```

Private Sub Command1_Click()
    Dim n As Integer
    n = 1
    While n <> -1
        n = InputBox("Enter number:", "While", "1")
        Print n
    Wend
End Sub
  
```


3. Run the project and click the button. An input box will appear. Enter 10 and press OK. This value will be printed on the form and the input box will appear again. This process will continue until you enter -1 in the input box.



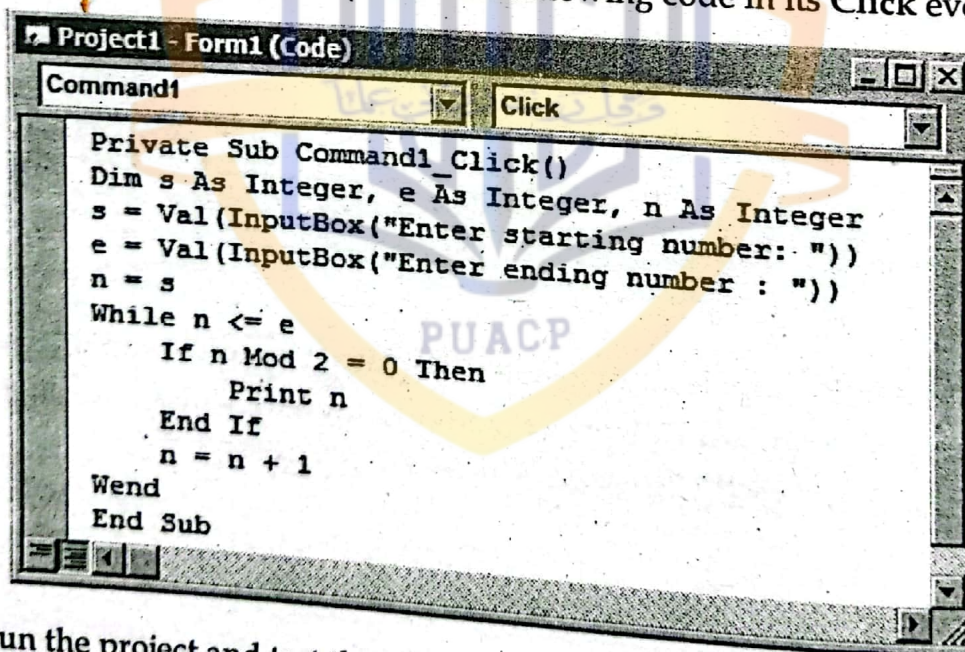
How It Works?

The above example is a sentinel-controlled loop. This example uses -1 as sentinel value to terminate the loop. The variable *n* is initialized to 1. First of all, the condition is evaluated and the control enters the body of the loop because the condition is true. The first statement in the body gets the input from the user and prints that value on the form. Then the control moves to the beginning of loop and the condition is evaluated again. This process continues until the user enters -1.

Program 6.4

Write a program that inputs starting and ending number from the user and then displays all even numbers in the given range using While loop.

1. Start a new Standard EXE project.
2. Place a button on the form.
3. Double click the button and add the following code in its Click event.

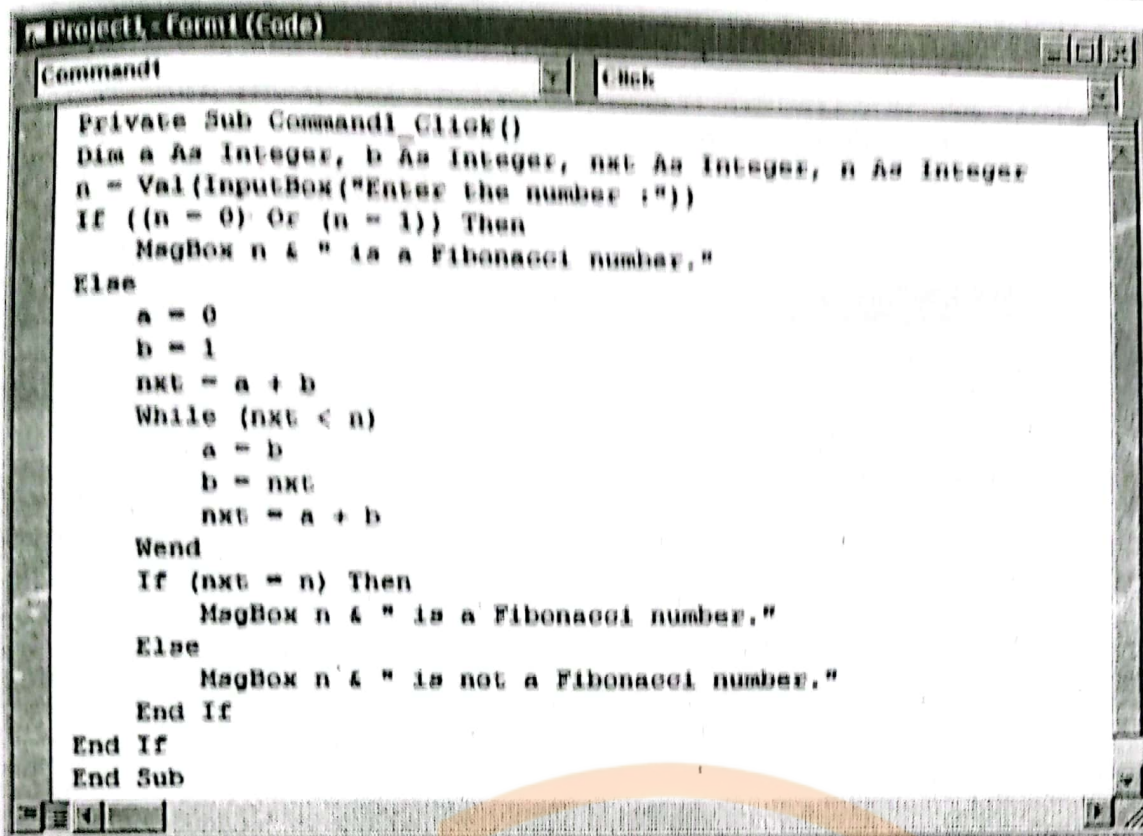


4. Run the project and test the working.

Program 6.5

Write a program that inputs a number and checks if it is a Fibonacci number or not. In Fibonacci sequence, sum of two successive terms gives the third term.

1. Start a new Standard EXE project.
2. Place a button on form.
3. Double click the button and add the following code in its Click event.

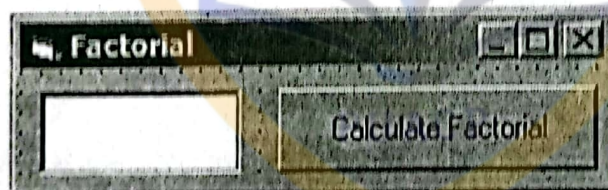


4. Run the project and check its working.

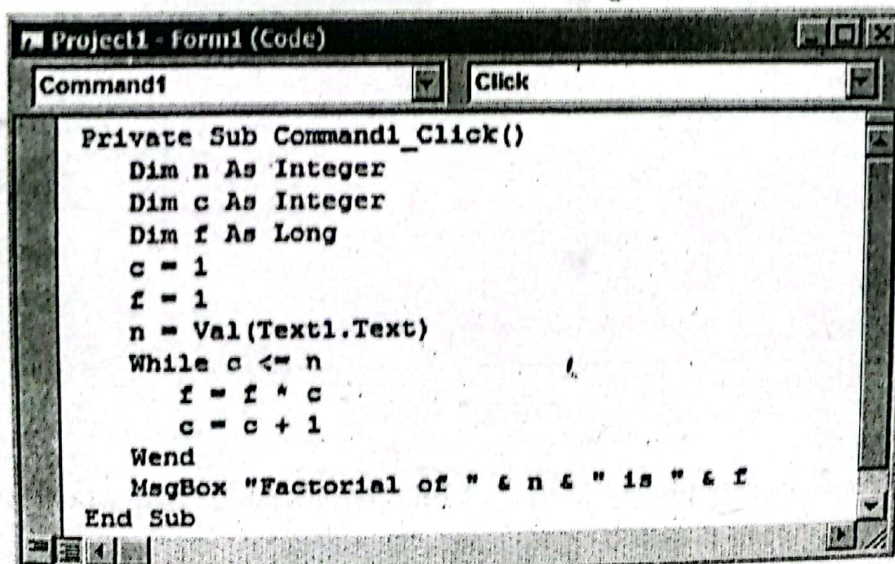
Program 6.6

Write a program that inputs a number from the user in textbox and displays factorial of that number.

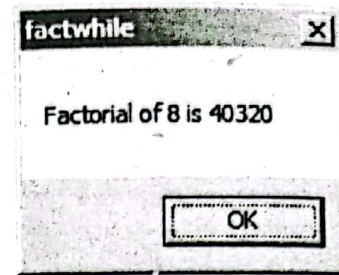
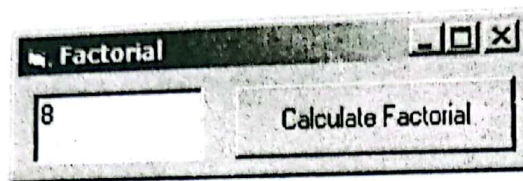
1. Start a new Standard EXE project.
2. Design the form as follows:



3. Double click the button and add the following code in its Click event.



4. Run the project.
5. Enter any positive number in text box and click the button.
6. Result will appear in the message box as follows:



6.3 Do...While...Loop

This loop is similar to while...wend loop. It executes one or more statements while the given condition is true.

Syntax

The syntax of Do...While loop is as follows:

Do While condition
Statement(s)

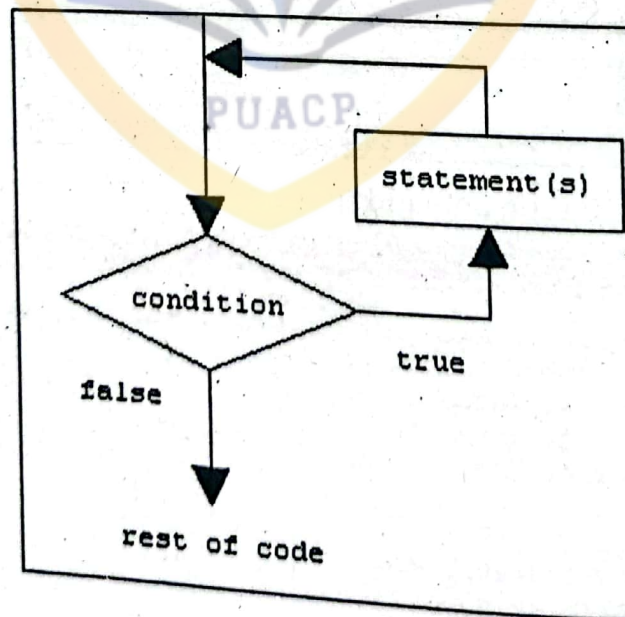
Loop

Working of Do...While Loop

First of all, the condition is evaluated. If it is true, the control enters the body of the loop and executes all statements. After executing the loop body, it again moves to the start of the loop and evaluates the condition again. This process continues as long as the condition remains true. The keyword **Loop** indicates the end of loop body.

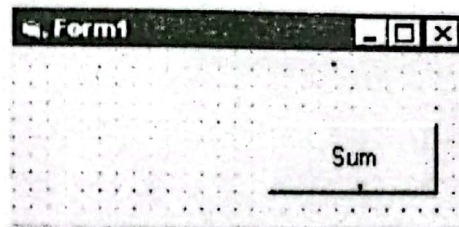
Flowchart

The flowchart of Do...While loop is as follows:

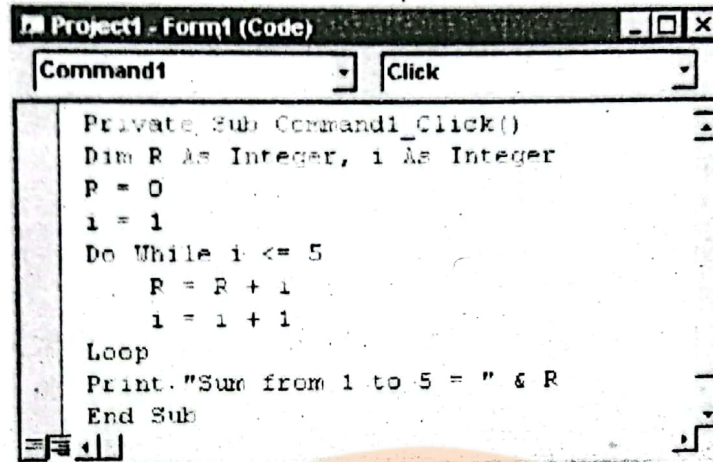


Program 6.7

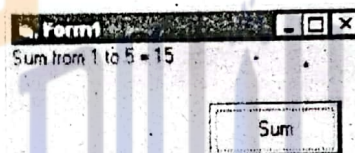
- Write a program that displays the sum of first five numbers using Do...While loop.
1. Start a new Standard EXE project.
 2. Design the form as follows:



3. Double click the button and add the following code in its Click event:



4. Run the project and click the button. The result will appear as follows:



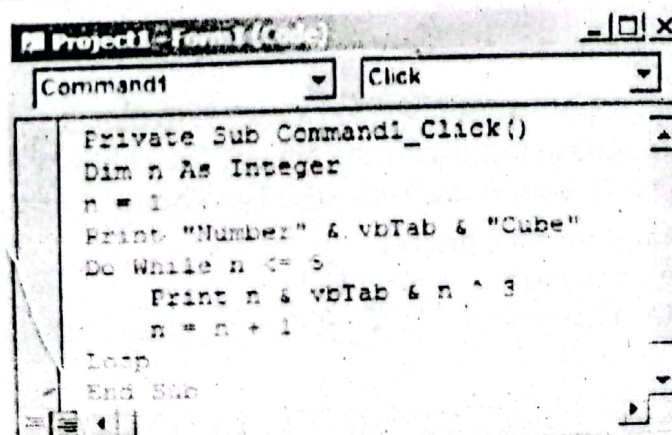
How It Works?

The above example uses two variables *i* and *R*. *R* is used to store the result and *i* is the counter variable and is initialized to 1. First of all, the condition is evaluated which is true. So the control enters the body of the loop. The value of counter variable is added to *R*. This process is repeated for five times. When the value of *i* becomes 6, the condition becomes false and the loop terminates. In the end, the final result is displayed on the form.

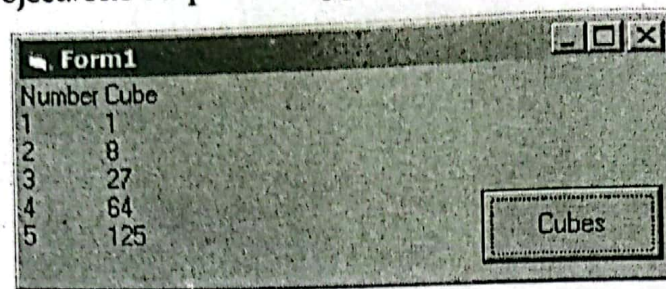
Program 6.8

Write a program that displays first five numbers with their cubes using Do-While loop.

1. Start a new Standard EXE project.
2. Place a button on the form.
3. Double click the button and add the following code in its Click event.



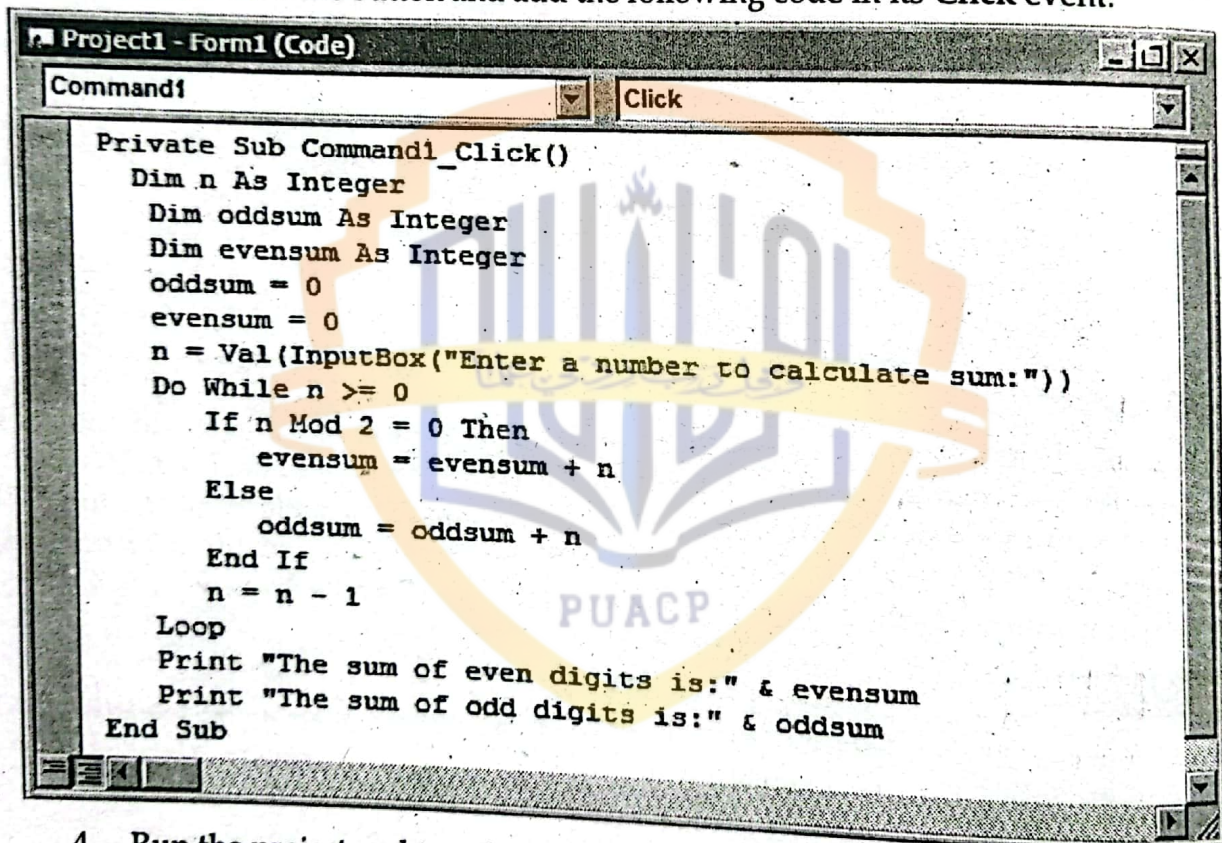
4. Run the project. The output will appear as follows:



Program 6.9

Write a program that inputs a number. It then displays the sum of all odd numbers and the sum of all even numbers from 1 to the number entered by the user. For example, if the user enters 5, the sum of odd numbers is 9 and sum of even numbers is 6.

1. Start a new Standard EXE project.
2. Place a button on the form.
3. Double click the button and add the following code in its Click event.

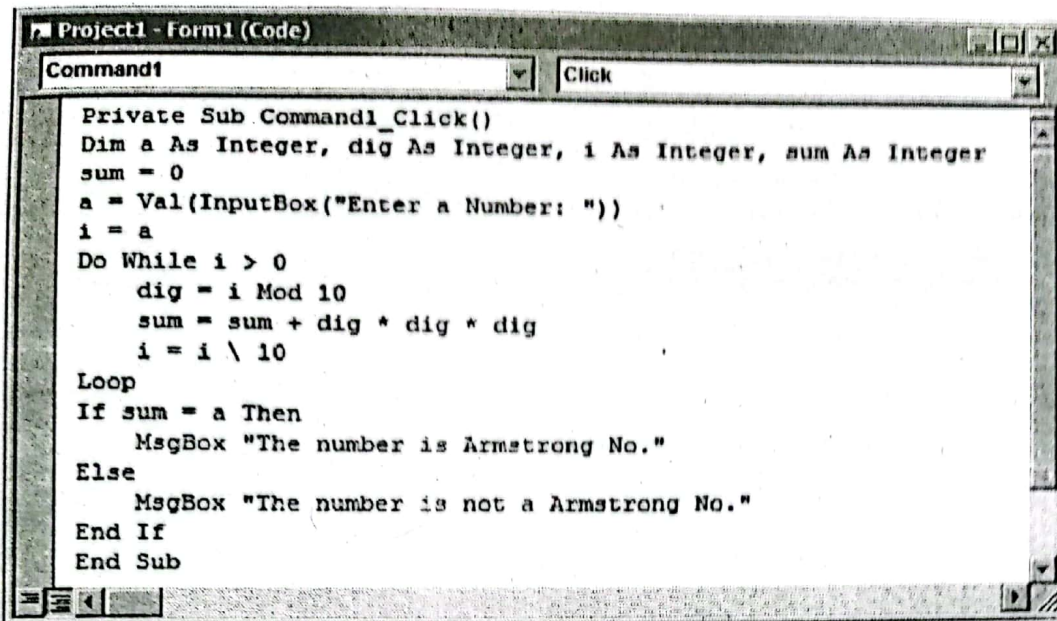


4. Run the project and test the working.

Program 6.10

Write a program that inputs a number and check whether it is an Armstrong number or not. A number is an Armstrong number if the sum of the cubes of its digits is equal to the number itself. For example, 371 is an Armstrong number since $3^3 + 7^3 + 1^3 = 371$.

1. Start a new Standard EXE project.
2. Place a button on the form.
3. Double click the button and add the following code in its Click event.

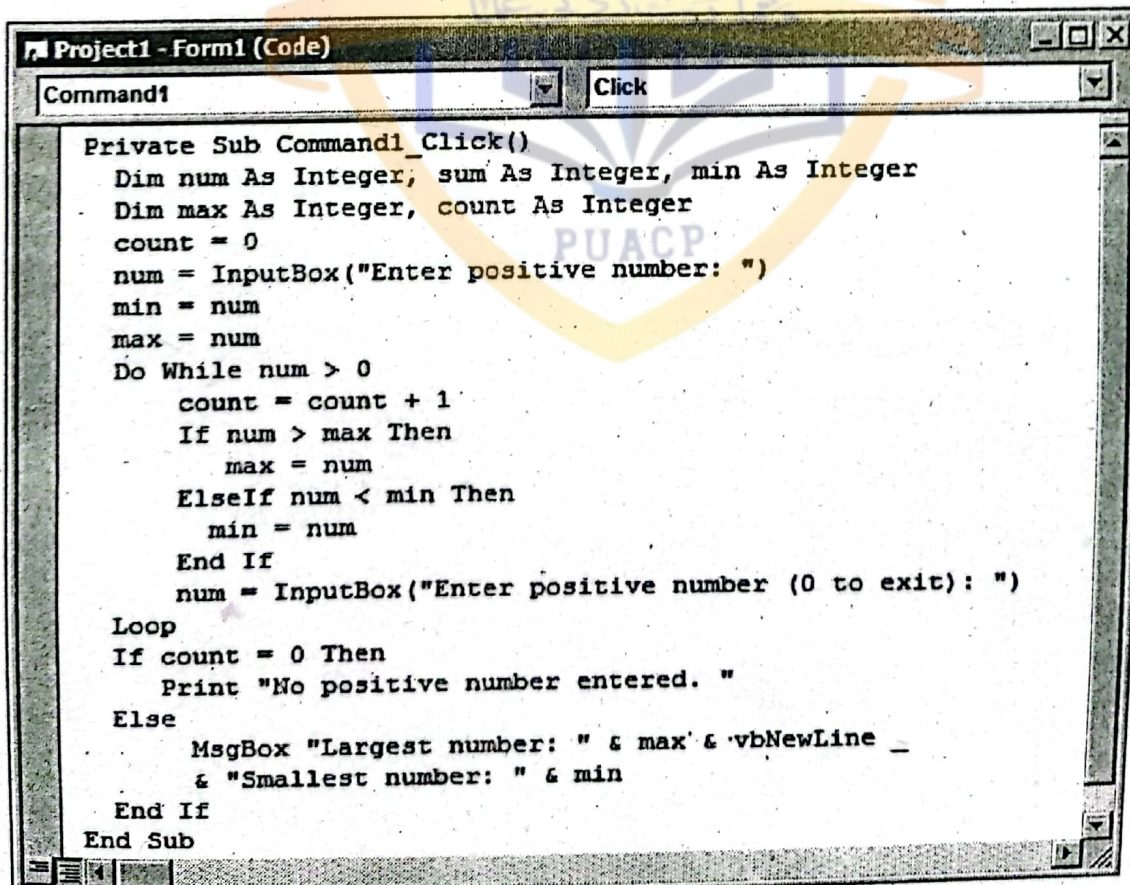


4. Run the project and test the working.

Program 6.11

Write a program that continuously inputs positive integer values from the user. The user enters a 0 to show that he has no more values to enter. The program should finally display the largest and smallest number entered.

1. Start a new Standard EXE project.
2. Place a button on the form.
3. Double click the button and add the following code in its Click event.

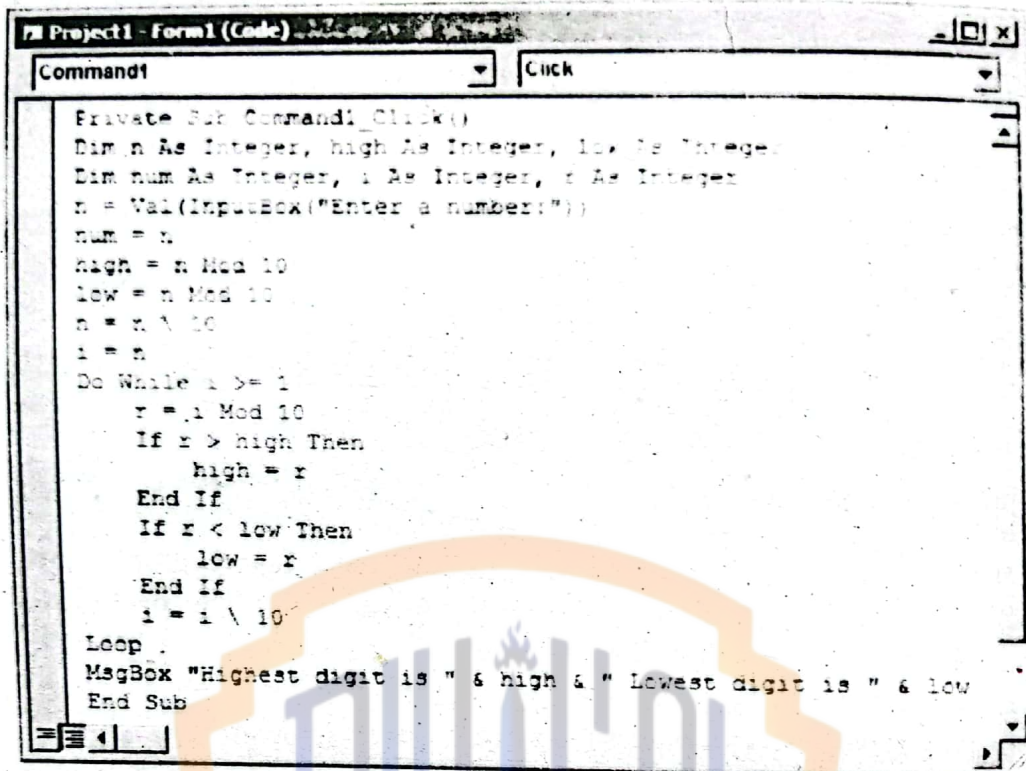


4. Run the project and test the working.

Example 6.12

Write a program that inputs a number from user and prints lowest and highest digit in the number. For example, if the user enters 219, the highest digit is 9 and lowest digit is 1.

1. Start a new Standard EXE project and place a button on the form.
2. Double click the button and add the following code in its Click event.



```

Private Sub Command1_Click()
    Dim n As Integer, high As Integer, low As Integer
    Dim num As Integer, i As Integer, r As Integer
    n = Val(InputBox("Enter a number:"))
    num = n
    high = n Mod 10
    low = n Mod 10
    i = n \ 10
    i = n
    Do While i >= 1
        r = i Mod 10
        If r > high Then
            high = r
        End If
        If r < low Then
            low = r
        End If
        i = i \ 10
    Loop
    MsgBox "Highest digit is " & high & " Lowest digit is " & low
End Sub

```

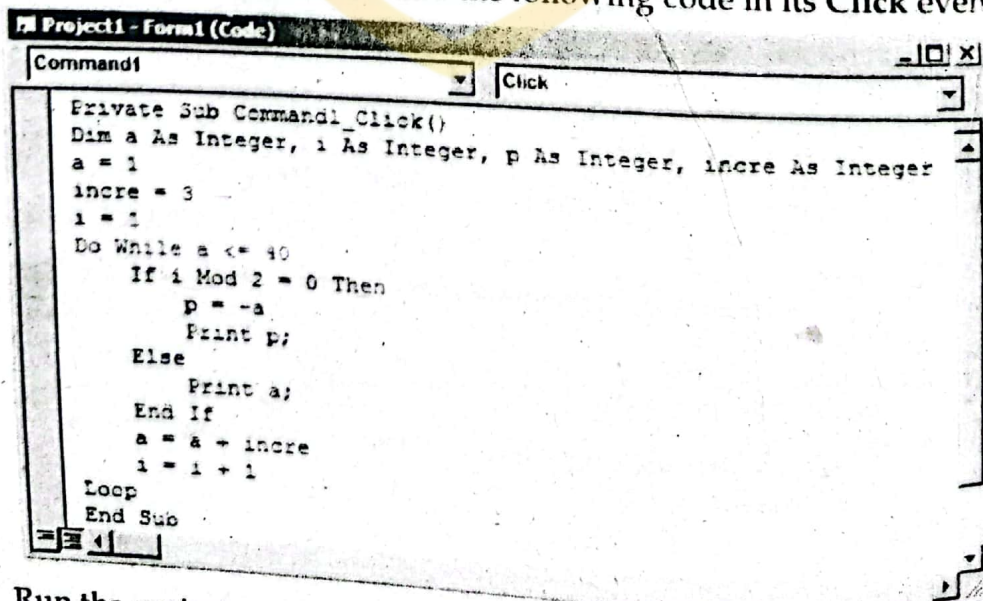
3. Run the project and test the working.

Example 6.13

Write a program to print the following series :

1 -4 7 -10 -40

1. Start a new Standard EXE project and place a button on the form.
2. Double click the button and add the following code in its Click event.



```

Private Sub Command1_Click()
    Dim a As Integer, i As Integer, p As Integer, incre As Integer
    a = 1
    incre = 3
    i = 1
    Do While a <= 40
        If i Mod 2 = 0 Then
            p = -a
            Print p;
        Else
            Print a;
        End If
        a = a + incre
        i = i + 1
    Loop
End Sub

```

3. Run the project and test the working.

6.4 Do...Until...Loop

This loop executes one or more statements while the given condition remains false.

Syntax

The syntax of this loop is as follows:

```
Do Until condition
    Statement(s)
```

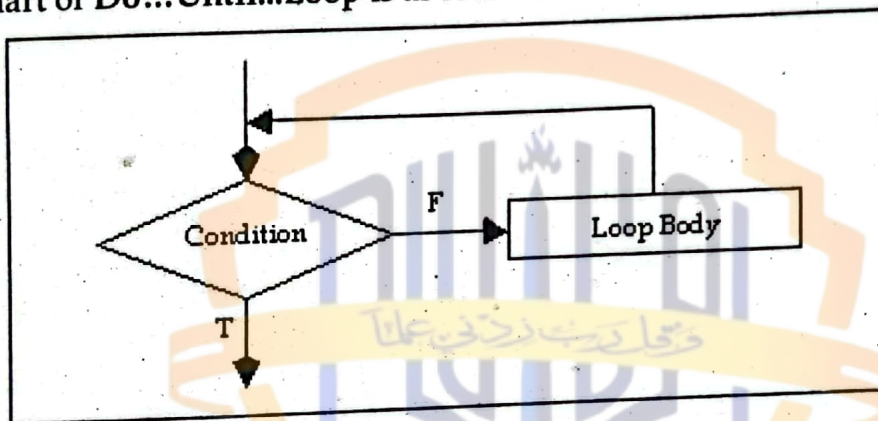
Loop

Working of Do...Until Loop

First of all, the condition is evaluated. If it is false, the control enters the body of the loop and executes all statements. After executing the loop body, it moves to the start of the loop and evaluates the condition again. This process continues as long as the condition remains false. The keyword Loop indicates the end of loop body.

Flowchart

The flowchart of Do...Until...Loop is as follows:



Program 6.14

Write a program that inputs a number from the user and finds its factorial.

1. Start a new Standard EXE project and place a button on the form.
2. Double the button and add the following code in its Click event:

```

Project1 - Form1 (Code)
Click
Command1

Private Sub Command1_Click()
    Dim n As Integer, c As Integer
    Dim f As Long
    c = 1
    f = 1
    n = Val(InputBox("Enter a number: "))
    Do Until c > n
        f = f * c
        c = c + 1
    Loop
    MsgBox "Factorial of " & n & " is " & f
End Sub
  
```

3. Run the project and check its working.

6.5 Do...Loop...While

This loop executes one or more statements while the given condition is true. This loop is similar to Do...while...Loop except that it first executes the loop body and then checks the condition. It means that the body of the loop is executed once even if the condition is false. So this loop should be used when you want to execute a statement at least once.

Syntax

The syntax of this loop is as follows:

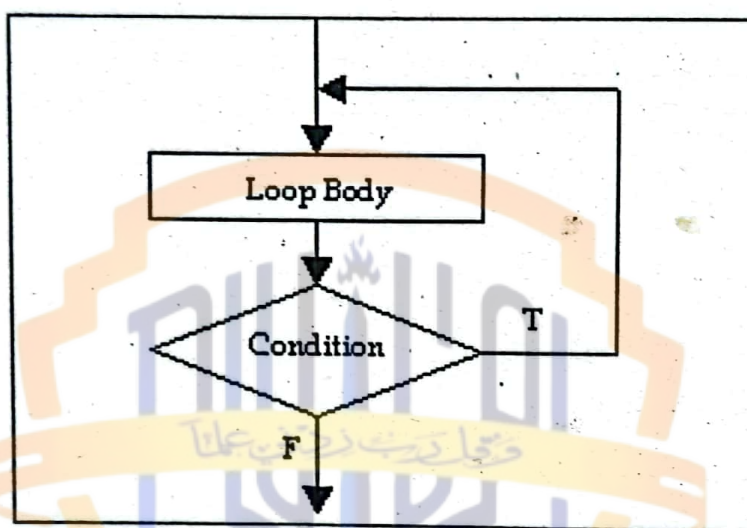
Do

Statement(s)

Loop While condition

Flowchart

The flowchart of Do...Loop...While is as follows:



Program 6.15

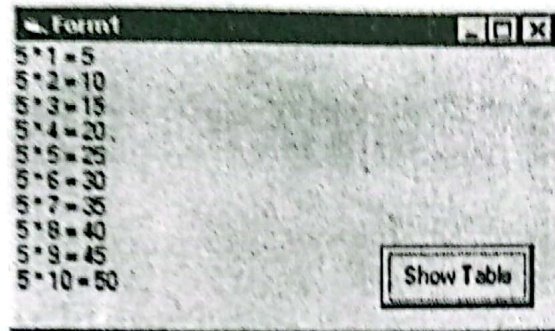
Write a program that gets a number from user and displays the table of that number.

1. Start a new Standard EXE project and place a button on the form.
2. Double click the button and add the following code in its Click event:

```

Project1 - Form1 (Code)
Command1 Click
Private Sub Command1_Click()
    Dim n As Integer, i As Integer
    n = InputBox("Enter a number:", "Table", "2")
    i = 1
    Do
        Print n & " * " & i & " = " & n * i
        i = i + 1
    Loop While i <= 10
End Sub
  
```

3. Run the project and click the button. An input box will appear.
4. Enter any number and click OK. The result will appear as follows:



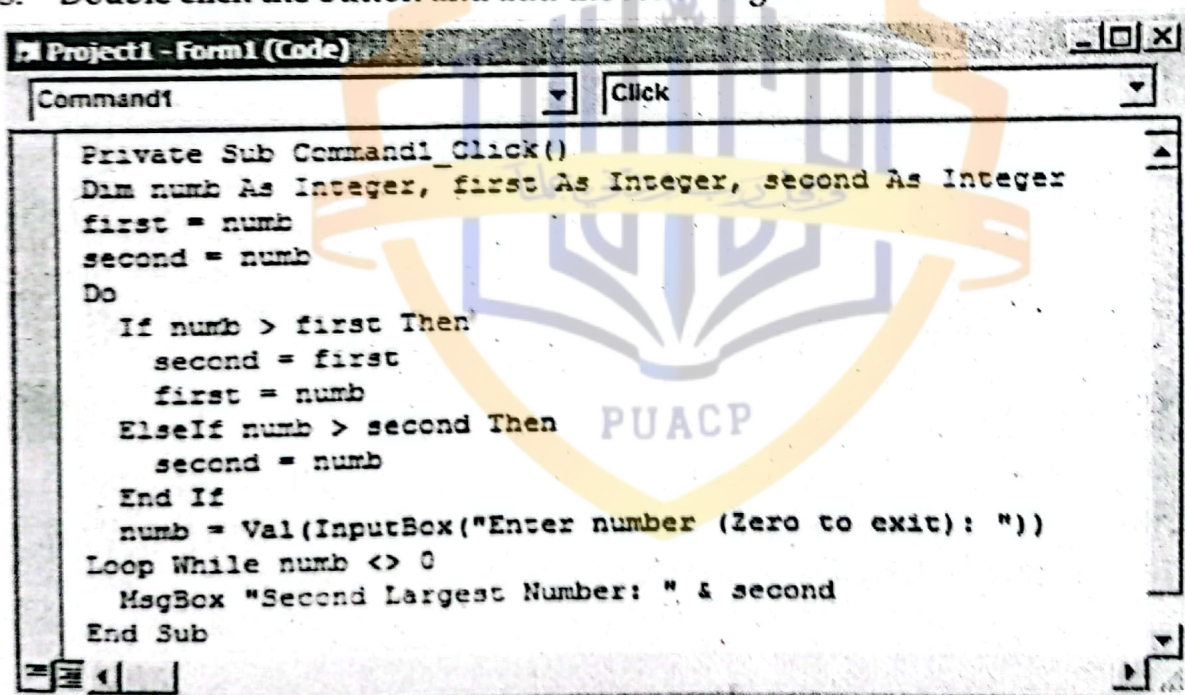
How It Works?

The above example is executed for ten times. Each time, the value of counter variable is different. So that output statement displays different result on the form.

Program 6.16

Write a program that continuously inputs positive integer values from the user. The user enters a zero to show that he has no more values to enter. The program should finally display the second largest number entered.

1. Start a new Standard EXE project.
2. Add a button on the form.
3. Double click the button and add the following code in its Click event.

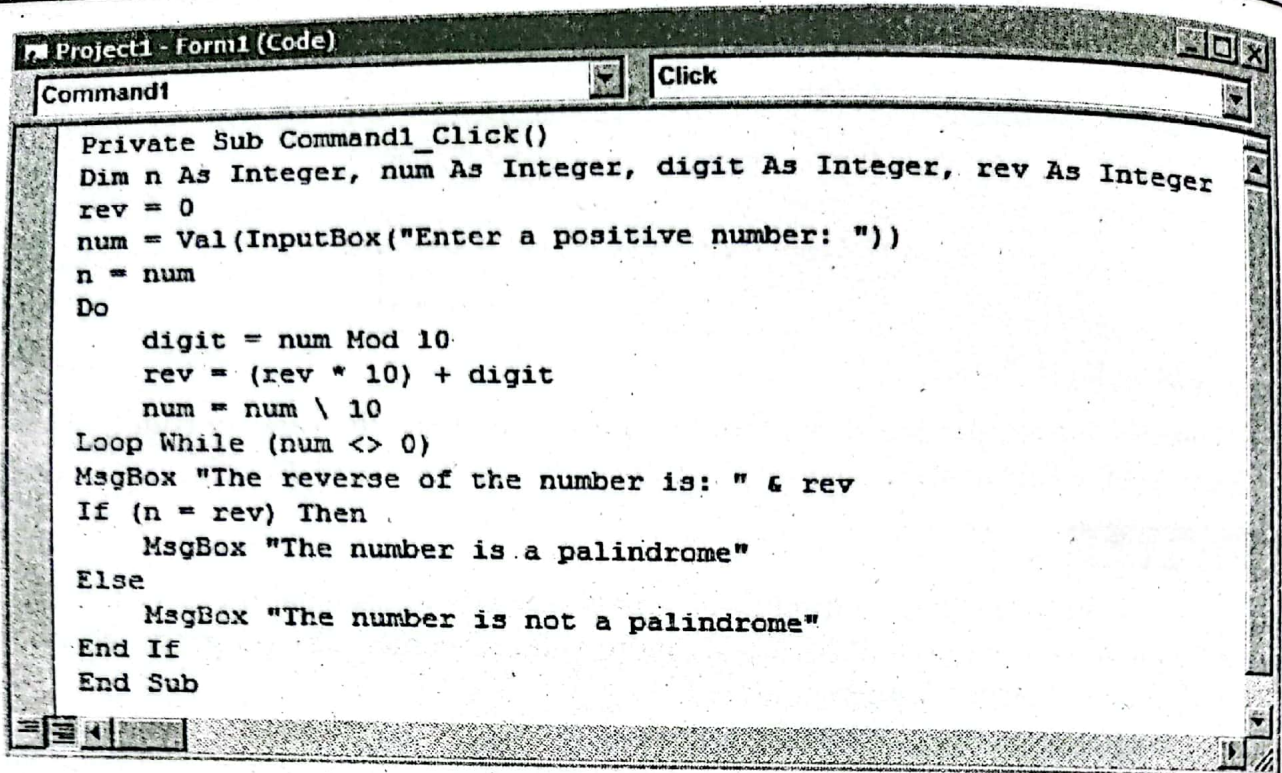


4. Run the project and test the working.

Program 6.17

Write a program that inputs a number whether it is a palindrome or not. A palindrome is a number that reads the same backwards as forwards such as 62526 and 4994.

1. Start a new Standard EXE project.
2. Add a button on the form.
3. Double click the button and add the following code in its Click event.



4. Run the project and test the working.

6.6 Do...Loop...Until

This loop executes one or more statements while the given condition is false. This loop is similar to Do...Until...Loop except that it first executes the loop body and then checks the condition. It means that the body of the loop is executed once even if the condition is true. So this loop should be used when you want to execute a statement at least once.

Syntax

The syntax of this loop is as follows:

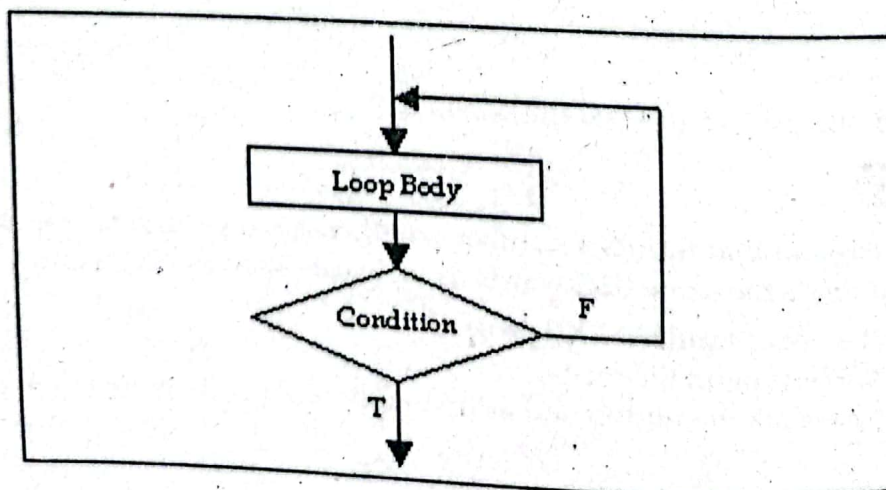
```

Do
    Statement(s)
Loop Until condition

```

Flowchart

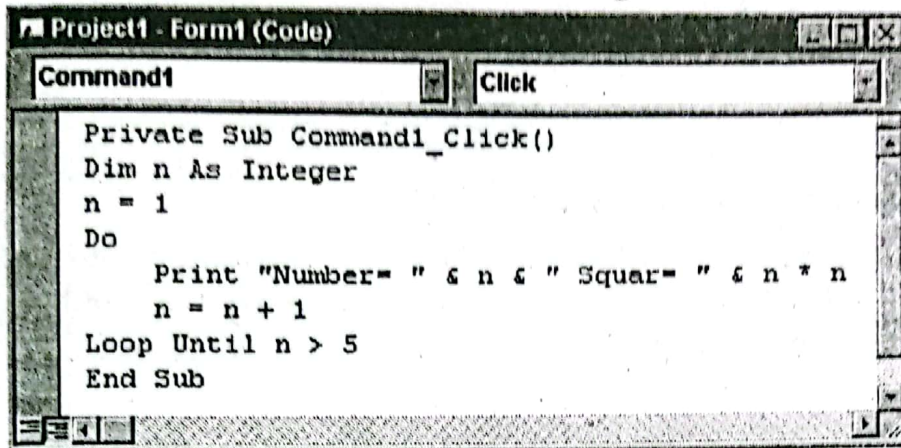
The flowchart of Do...Loop...Until is as follows:



Program 6.18

Write program to display first five numbers and their squares using Do...Loop...Until.

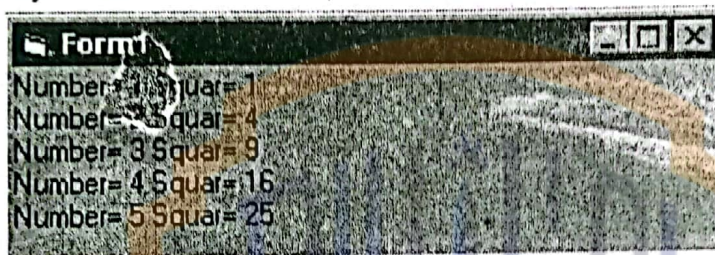
1. Start a new Standard EXE project and place a button on the form.
2. Double click the button and add the following code in its Click event:



```

Project1 - Form1 (Code)
Command1 Click
Private Sub Command1_Click()
    Dim n As Integer
    n = 1
    Do
        Print "Number= " & n & " Squar= " & n * n
        n = n + 1
    Loop Until n > 5
End Sub
  
```

3. Run the project and click the button. The result will appear as follows:

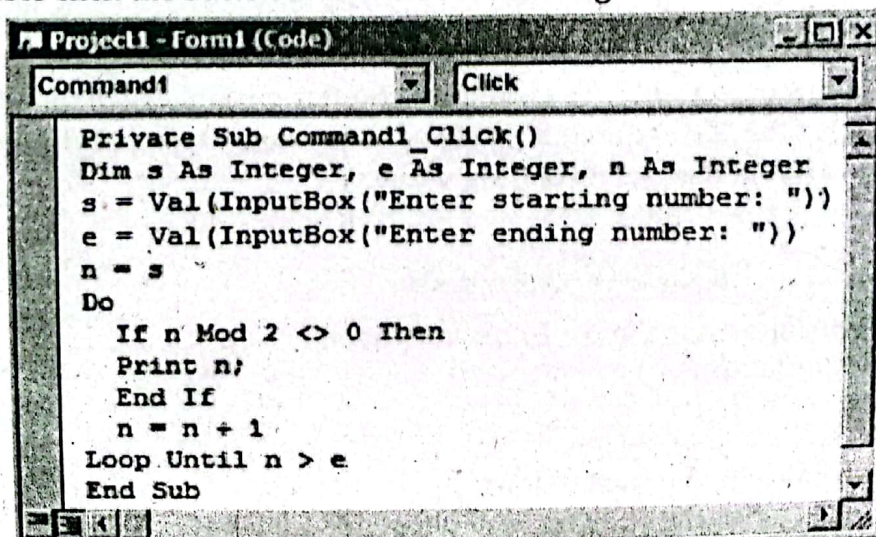
**How It Works?**

The above example uses *n* as counter variable. Its value is changed from 1 to 5. The loop displays the value of *n* and its square. When the value of *n* becomes 6, the condition becomes true and the loop terminates.

Program 6.19

Write a program that inputs starting and ending number from the user and then displays all odd numbers in the given range using Do...Loop...Until.

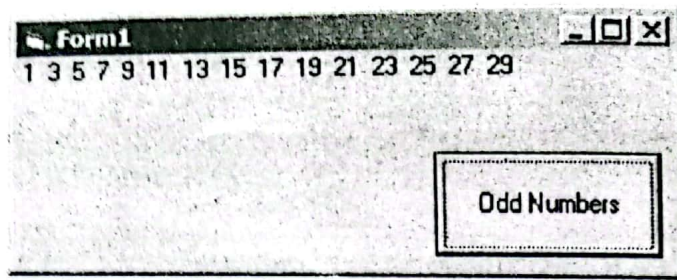
1. Start a new Standard EXE project and place a button on the form.
2. Double click the button and add the following code in its Click event.



```

Project1 - Form1 (Code)
Command1 Click
Private Sub Command1_Click()
    Dim s As Integer, e As Integer, n As Integer
    s = Val(InputBox("Enter starting number: "))
    e = Val(InputBox("Enter ending number: "))
    n = s
    Do
        If n Mod 2 <> 0 Then
            Print n;
        End If
        n = n + 1
    Loop Until n > e
End Sub
  
```


3. Run the project and click the button.
4. Enter 1 as starting and 30 as ending values in input boxes. The result will appear:



6.7 Pretest and Posttest in Loops

A pretest loop is a loop in which the condition is evaluated before the body of the loop. The execution of statements in the loop body depends on the result of the condition. In this type of loop, the statements in the loop body may never be executed.

Example

Following is an example of pretest condition. The statements in the body of loop cannot be executed if the condition is false.

```
n = 0
While n = 0
    Statements in loop
Wend
```

A posttest loop is a loop in which the condition is evaluated after the body of the loop. In this type of loop, the statements in the loop body are executed at least once.

Example

Following is an example of posttest condition. The statements in the body of loop will be executed at least once even if the condition is false.

```
Do
    Statements in loop
Loop while n = 0
```

6.8 For...Next Loop

For...Next loop is used to execute a statement or set of statements for the specified number of times. This loop is more useful when it is already known how many times the loop will be executed. This loop depends on the value of a variable to repeat the statements. This variable is called counter variable or control variable. The loop specifies the starting and ending values of counter variable. The value of counter variable is changed according to the specified value each time the statement or set of statements is executed. The loop terminates when the counter variable reaches its ending value.

Syntax

The syntax of For...Next loop is as follows:

```
For CounterVar = StartVar To EndVar [Step n]
    statement(s)
Next
```

For It indicates the start of loop.
CounterVar It indicates the counter variable on the loop depends.

StartVar
EndVar
Step

It indicates the starting value of the counter variable.

It indicates the ending value of the counter variable.

It is used to change the value of counter variable each time the statement or set of statement is executed. It is optional. The value of counter variable is increased by 1 if it is not used.

It indicates the change that will occur in the counter variable each time the statement or set of statement is executed.

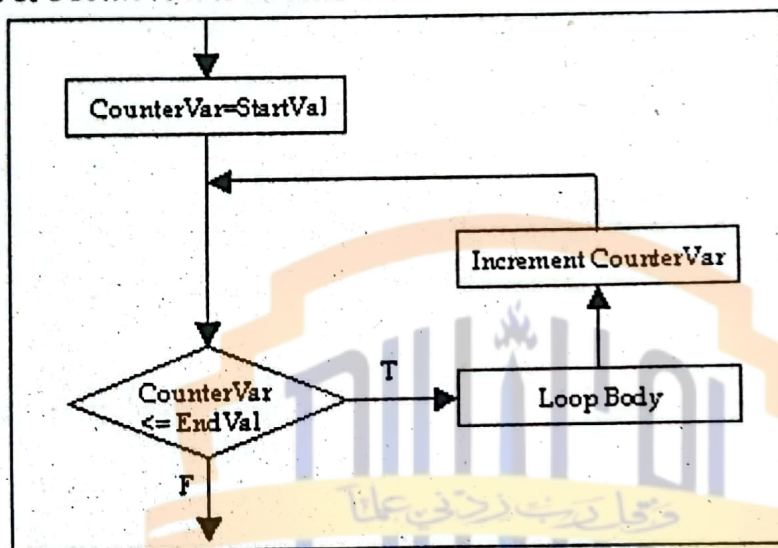
statements(s)
Next

It indicates the statement or set of statements to be repeated by the loop.

It indicates the end of loop. The statements written between For and Next are repeated by the loop.

Flowchart

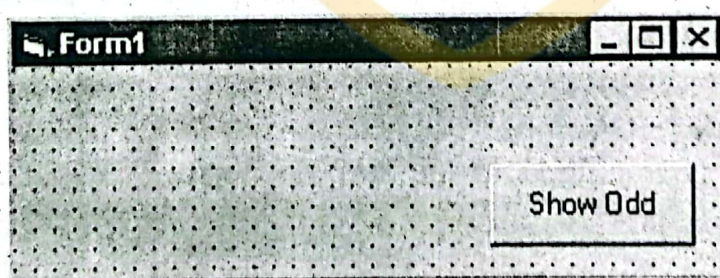
The flowchart of For...Next is as follows:



Program 6.20

Write a program that displays first five odd numbers using For...Next loop.

1. Start a new Standard EXE project.
2. Design the form as follows:

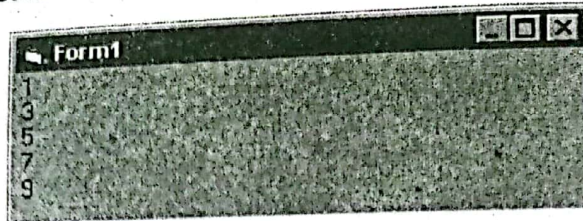


3. Double click the button and add the following code in its Click event:

```

Project1 - Form1 (Code)
Command1 Click
Private Sub Command1_Click()
    Dim n As Integer
    For n = 1 To 10 Step 2
        Print n
    Next
End Sub
  
```


4. Run the project and click the button. The result will appear as follows:



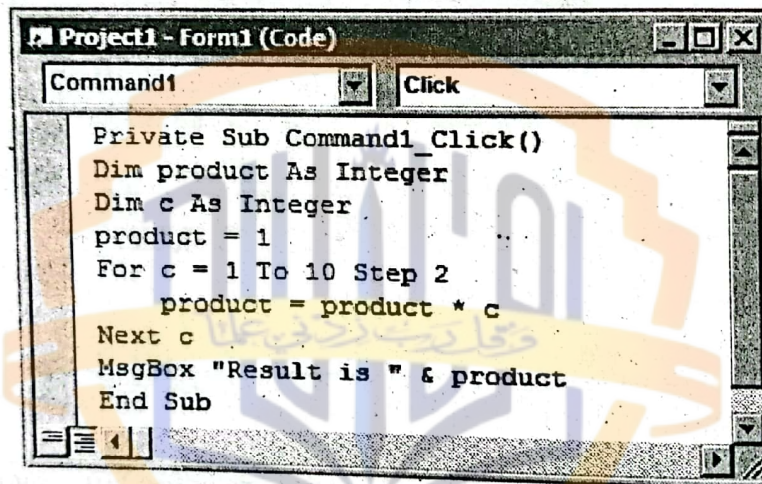
• How It Works?

In the above example, n is the counter variable that is initialized to 1. After each iteration, the value of n is incremented by 2. The loop is executed for five times. It terminates when the value of n becomes 11.

Program 6.21

Write a program that calculates and displays product of all odd numbers from 1 to 10.

1. Start a new Standard EXE project.
2. Place a button on the form.
3. Double click the button and add the following code in its Click event.

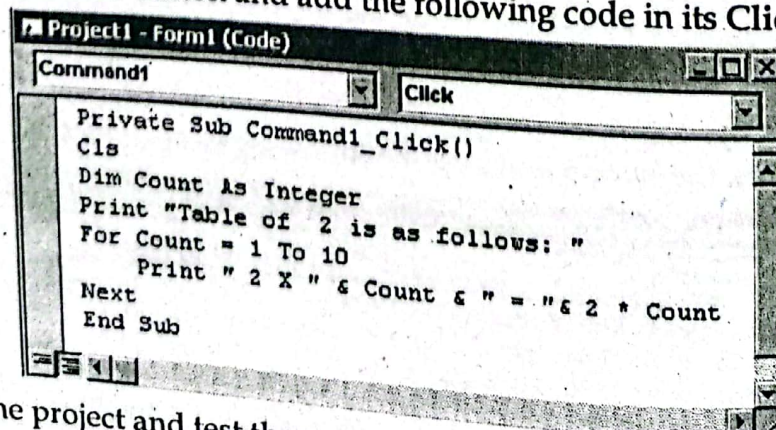


4. Run the project and test the working.

Program 6.22

Write a program that displays table of 2 using For Next loop.

1. Start a new Standard EXE project.
2. Place a button on the form.
3. Double click the button and add the following code in its Click event.



4. Run the project and test the working.

Program 6.23

Write a program that will prompt the user for table number and limit up to which the table should be printed using inputbox function. The table should be printed on the form as per the user inputs.

1. Start a new Standard EXE project.
2. Place a button on the form.
3. Double click the button and add the following code in its Click event.

```

Private Sub Command1_Click()
    Dim n As Integer, limit As Integer
    n = Val(InputBox("Enter table number: "))
    limit = Val(InputBox("Enter table limit: "))
    For i = 1 To limit
        Print n & " * " & i & " = " & i * n
    Next
End Sub

```

4. Run the project and test the working.

Program 6.24

Write a program that finds the sum of squares of the integers from 1 to n where n is a positive integer value entered by the user (i.e. $\text{Sum} = 1^2 + 2^2 + 3^2 + \dots + n^2$).

1. Start a new Standard EXE project.
2. Place a button on the form.
3. Double click the button and add the following code in its Click event.

```

Private Sub Command1_Click()
    Dim No, Count, Sum As Integer
    Sum = 0
    No = InputBox("Enter any number: ", "Number", "0")
    For Count = 1 To No
        Sum = Sum + (Count * Count)
    Next
    MsgBox "Sum of sq. of integers up to " & No & " = " & Sum
End Sub

```

4. Run the project and test the working.

Example 6.25

Write a program that takes n numbers as input. It outputs the frequency of positive and negative numbers. Supposes the user inputs 12 in variable n and enters the following numbers:

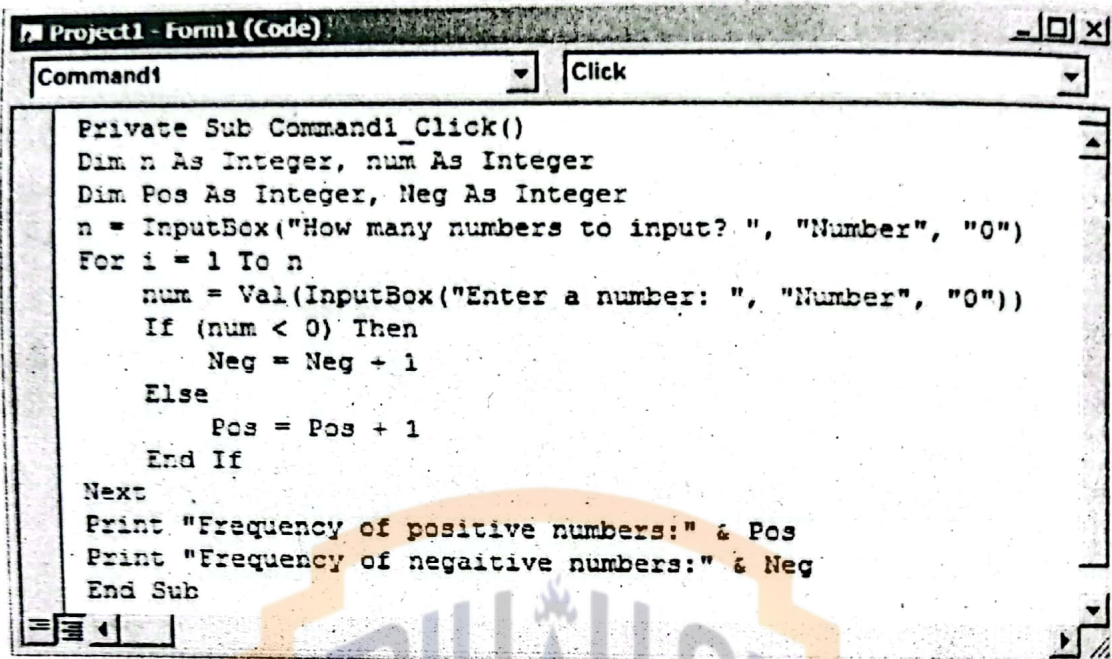
40 0 -1 59 -37 283 61 387 -91 -103 341 523

The result should be as follows:

Frequency of positive numbers: 8

Frequency of negative numbers: 4

1. Start a new Standard EXE project and place a button on the form.
2. Double click the button and add the following code in its Click event.



```

Private Sub Command1_Click()
    Dim n As Integer, num As Integer
    Dim Pos As Integer, Neg As Integer
    n = InputBox("How many numbers to input? ", "Number", "0")
    For i = 1 To n
        num = Val(InputBox("Enter a number: ", "Number", "0"))
        If (num < 0) Then
            Neg = Neg + 1
        Else
            Pos = Pos + 1
        End If
    Next
    Print "Frequency of positive numbers:" & Pos
    Print "Frequency of negative numbers:" & Neg
End Sub

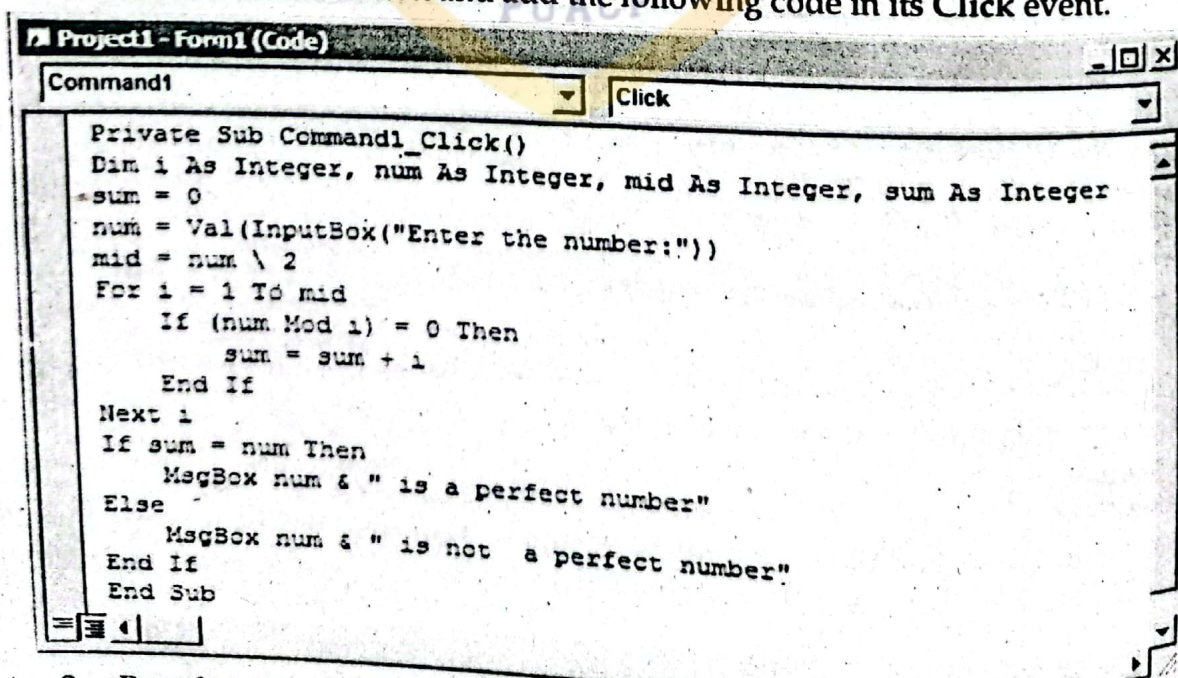
```

3. Run the project and test the working.

Program 6.26

Write a program that inputs a number and checks whether it is a perfect number or not. A perfect number is a number that is numerically equal to the sum of its divisors. For example, 6 is a perfect number because the divisors of 6 are 1, 2, 3 and $1+2+3=6$.

1. Start a new Standard EXE project and place a button on the form.
2. Double click the button and add the following code in its Click event.



```

Private Sub Command1_Click()
    Dim i As Integer, num As Integer, mid As Integer, sum As Integer
    sum = 0
    num = Val(InputBox("Enter the number:"))
    mid = num \ 2
    For i = 1 To mid
        If (num Mod i) = 0 Then
            sum = sum + i
        End If
    Next i
    If sum = num Then
        MsgBox num & " is a perfect number"
    Else
        MsgBox num & " is not a perfect number"
    End If
End Sub

```

3. Run the project and test the working.

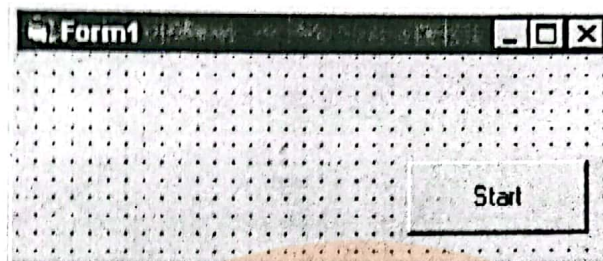
6.9 Early Termination of Loops

Normally, the termination of the loop depends on the given condition. But you can terminate a loop earlier, without looking at the condition. **Exit For** statement is used in the body of **For...Next** loop to terminate it abnormally. **Exit Do** statement is used with **Do** loops for the same purpose.

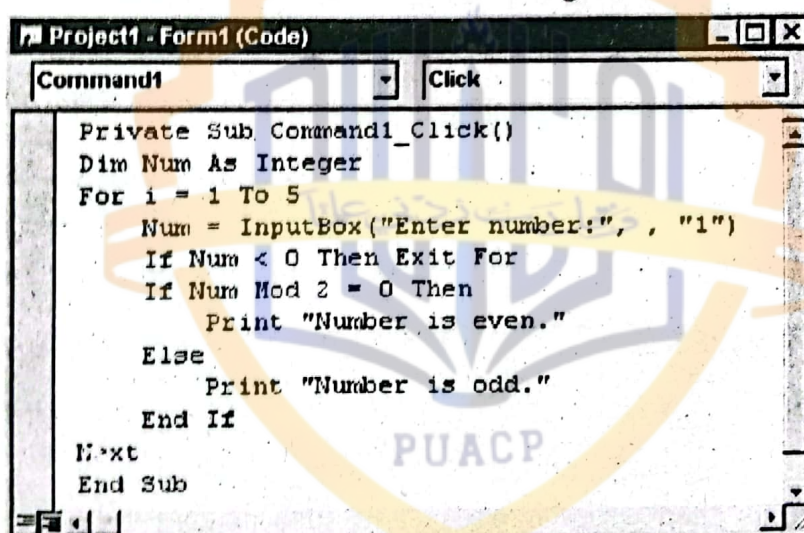
Program 6.27

Write a program that gets five numbers from the user and displays whether the number is even or odd. But if user enters a negative number, it is terminated immediately.

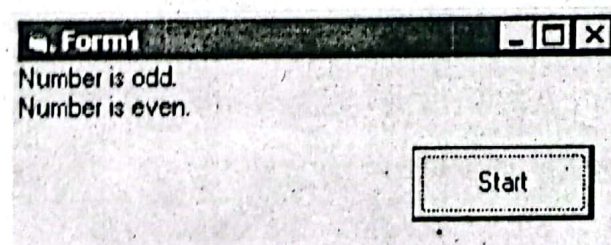
1. Start a new Standard EXE project.
2. Design the form as follows:



3. Double click the button and add the following code in its Click event:



4. Run the project and click the button. An input box will appear.
5. Enter any number and click OK. The result will appear as follows:

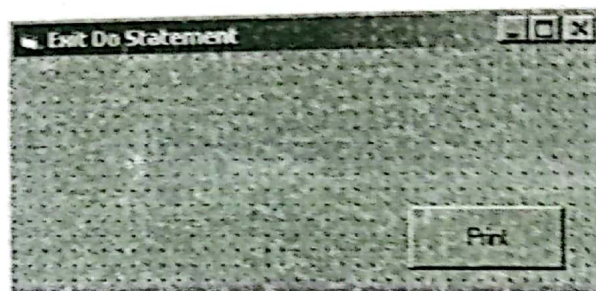


How It Works?

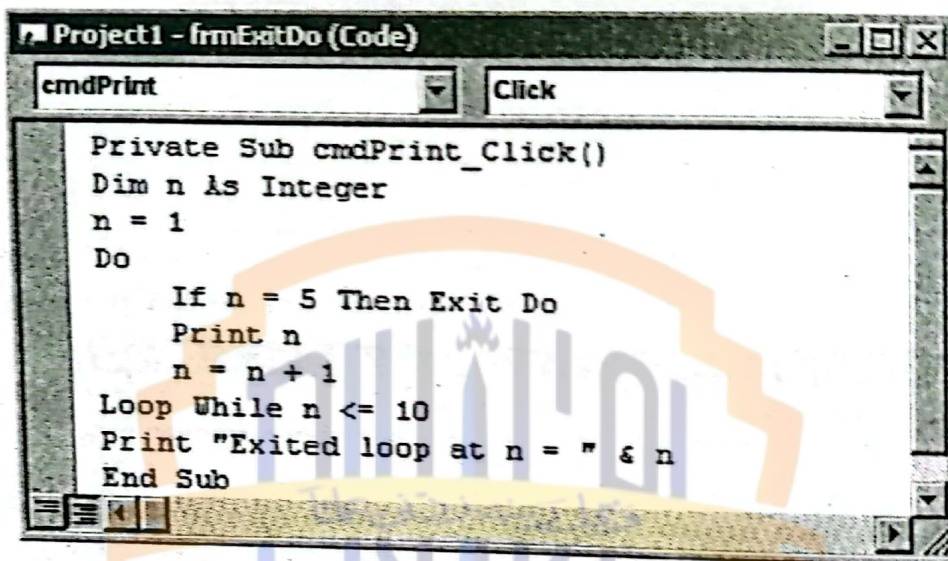
The loop in the above example will execute for five times if the user enters positive numbers. But if the input is negative, it will terminate immediately. The above figure indicates that first two inputs were positive numbers and third input was negative. Similarly, **Exit Do** statement can be used to terminate any **Do** loop before the normal termination.

Program 6.28

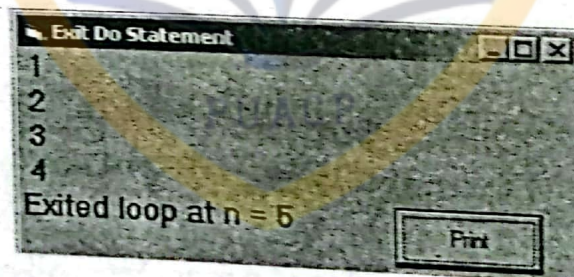
1. Start a new Standard EXE project.
2. Design the form as follows:



3. Double click the button and add the following code in its Click event:



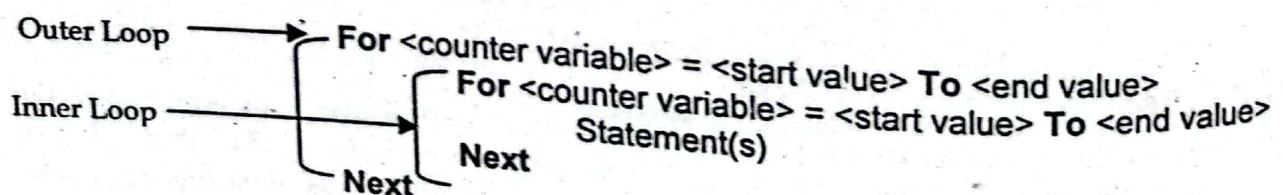
4. Run the project and click the button. The result will appear as follows:

**6.10 Nested Loops**

A loop within a loop is called nested loop. In nested loops, the inner loop is executed completely with each change in the value of counter variable of outer loop.

Syntax

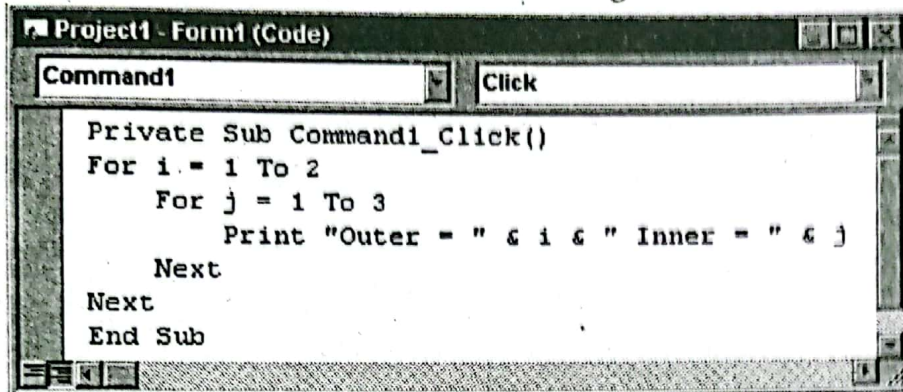
The syntax of nested loop is as follows:



Program 6.29

Write a program that uses a simple nested loop to display a message.

1. Start a new Standard EXE project and place a button on the form.
2. Double click the button and add the following code in its Click event:



```

Project1 - Form1 (Code)
Command1 Click
Private Sub Command1_Click()
    For i = 1 To 2
        For j = 1 To 3
            Print "Outer = " & i & " Inner = " & j
        Next
    Next
End Sub

```

3. Run the project and click the button. The result will appear as follows:



```

Form1
Outer = 1 Inner = 1
Outer = 1 Inner = 2
Outer = 1 Inner = 3
Outer = 2 Inner = 1
Outer = 2 Inner = 2
Outer = 2 Inner = 3

```

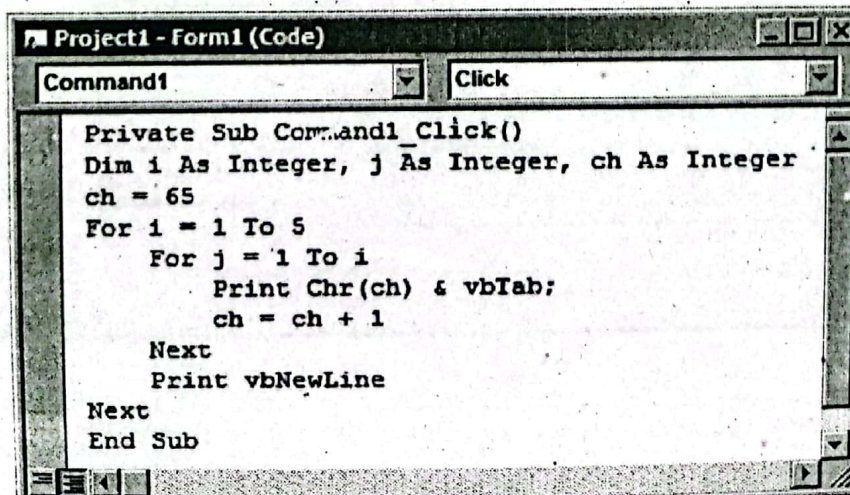
How It Works?

In the above example, the **outer loop** is executed for two times. With each execution of the outer loop, the **inner loop** executes completely for three times. First of all, the counter variable of outer loop **i** is 1. When the control moves to inner loop, its counter variable **j** changes from 1 to 3. When inner loop finishes after executing the output statement for three times, the control moves at the **beginning** of the outer loop. In this way, the output statement is actually executed for six times.

Program 6.30

Write a program that displays the **triangle of alphabets**:

1. Start a new Standard EXE project and place a button on the form.
2. Double click the button and add the following code in its Click event:



```

Project1 - Form1 (Code)
Command1 Click
Private Sub Command1_Click()
    Dim i As Integer, j As Integer, ch As Integer
    ch = 65
    For i = 1 To 5
        For j = 1 To i
            Print Chr(ch) & vbTab;
            ch = ch + 1
        Next
        Print vbNewLine
    Next
End Sub

```


3. Run the project and click the button. The result will appear as follows:

Program 6.31

Write a program that displays the square of dollar signs consisting of five rows and five columns.

1. Start a new Standard EXE project.
2. Place a button on the form.
3. Double click the button and add the following code in its Click event:

```

Private Sub Command1_Click()
    Dim m As Integer, n As Integer
    For m = 1 To 5
        For n = 1 To 5
            Print "$" & vbTab;
        Next
        Print vbNewLine
    Next
End Sub
  
```

4. Run the project and click the button. The result will appear as follows:

Programming Exercise

1. Write a program to print integers in a specified range. Make every alternate integer negative in output.
2. Write a program that gets two strings from the user and counts the number of times the second string appears in the first string.
3. Write a program that gets a century from the user and displays all leap years in that century.
4. Write a program to get an integer from the user and display whether it is prime or not.
5. Write a program to get starting and ending point from the user and display all even numbers in the given range.
6. Write a program to display counting from 1 to 10.
7. Write a program to display back counting from 10 to 1.
8. Write a program to display first five numbers with their squares and cubes.
9. Write a program that inputs starting and ending numbers and option from the user. If the option is "E", it displays all even numbers in the given range and their sum. If the option is "O", it displays all odd numbers in the given range and their sum.
10. Write a program that inputs a number from user and displays all fibonacci numbers up to that number. In Fibonacci sequence, sum of two successive terms gives the third term.
11. Write a program to display the sum of the following series:
 $1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{6} + \dots + \frac{1}{100}$.
12. Write a program that gets two numbers from the user and displays the result of first number raise to the power second number.
13. Write a program that inputs a number from the user and displays n terms of the Fibonacci series. For example, if the user enters 10, it displays 10 terms of the series.
14. Write a program that inputs the number of students in the class. It then inputs the marks of these students and displays the highest and second highest marks.
15. Write a program that inputs an integer and displays the sum of its digits. For example, the program should display 9 if the user enters 135.
16. Write a program that inputs numbers until the user enters a negative number. The program calculates the average, maximum and minimum of all positive numbers.
17. Write a program to print the following series :
1 4 7 10 40
18. Write a program that inputs a number and displays whether it is a perfect square or not.
19. Write a program that inputs two numbers and displays their greatest common divisor (GCD) and least common multiple (LCM).
20. Write a program that inputs a table number, starting point and ending point. It then displays the table.
21. Write a program that adds 3, 6, 9, 12 ... 99 and displays the result.
22. Write a program that inputs a positive integer and displays the prime factors of the number. The prime factors of a positive integer are the prime numbers that divide into that integer exactly, without leaving a remainder. For example, prime factors of 24 are 2, 2, 2 and 3, whereas prime factors of 35 are 5 and 7.
23. Write a program that inputs a number from the user and displays all prime numbers up to the entered number.

Important Note: The solutions of all questions are available at the end of the book.

Short Questions

Q.1. What is loop? What are two uses of loop?

A control structure that executes a statement or number of statements repeatedly is known as loop. Loops can be used to execute a statement or number of statements for a specified number of times. Loops can also be used to access a sequence of values such as 1,2,3 so on.

Q.2. What is loop? What are two uses of loop?

A control structure that executes a statement or number of statements repeatedly is known as loop. Loops can be used to execute a statement or number of statements for a specified number of times. Loops can also be used to access a sequence of values such as 1,2,3 so on.

Q.3. What is iteration?

One complete cycle of a loop is known as an iteration.

Q.4. Explain the difference between pretest and posttest loop with example.

A pretest loop is a loop in which the condition is evaluated before the body of the loop. The execution of statements in the loop body depend on the result of the condition. In this type of loop, the statements in the loop body may never be executed. The example of Do...While...Loop statement for a posttest loop is as follows:

```
intNum=1
Do While intNum<=5
    MsgBox(intNum)
    intNum=intNum+1
Loop
```

A posttest loop is a loop in which the condition is evaluated after the body of the loop. In this type of loop, the statements in the loop body are executed at least once. Pretest loops are more commonly used. The example of the Do...Loop...While statement for a posttest loop is as follows:

```
intNum=1
Do
    MsgBox(intNum)
    intNum=intNum+1
Loop While intNum<=3
```

Q.5. Can Do Until loop be written as pretest or posttest?

A loop that iterates while a condition remains false is known as a Do Until loop. The Do Until loop can be written as either a pretest or a posttest loop. The general format of a pretest Do Until loop:

```
Do Until Condition
    Statements
Loop
```

The general format of a posttest Do Until loop:

```
Do
    Statements
Loop Until Condition
```

Q.6. Describe the syntax and operation of the For-Next loop.

The general form of the For-Next loop follows

```
For CounterVar= StartVal To EndVal [Step n]
    statements
Next
```


This loop depends on the value of a variable to repeat the statements. This variable is called counter variable. This loop specifies the starting and ending values of counter variable. The value of counter variable is changed according to the specified value each time the statements or set of statements is executed. The loop terminates when the counter variable reaches its ending value.

Q.7. Describe the syntax and operation of Do While-Loop and Do-Loop Until.

The general form of the Do While-Loop is:

```
Do While condition
  statements
Loop
```

The general syntax of the Do-Loop Until is:

```
Do
  statements
Loop Until condition
```

The condition is a logical expression. The Do While-Loop repeats the statements inside the loop as long as condition is true. The Do-Loop Until repeats the statements as long as condition is false.

Q.8. What is counter variable?

A counter variable is a variable that is used to control the number of iteration of a loop. It is a numeric variables that is typically assigned a beginning value. The value of counter variable is updated in each iteration by incrementing or decrementing its value. The statement that updates a counter variable is placed within the loop body.

Q.9. Explain the differences between a Do...Loop and a For...Next loop.

For...Next loop is used when the number of iterations are already known. The loop index determines the number of times the statements are executed. A Do...Loop terminates the loop only when a specified condition is satisfied. It is used when the exact number of iterations is unknown.

Q.10. What is the difference between Do...While...Loop and Do...Loop...While?

In Do...While...Loop, the relational condition is checked at the top of the loop body. In Do...Loop...While loop, the condition is checked after the loop body.

Q.11. When should you use Do...While...loop?

The Do...While...Loop should be used when the statement should be executed as long as the given condition is true. It can be written as a pretest or posttest loop. The pretest Do...While...Loop does not execute the statement if the condition is false from the beginning. The posttest loops executes the statement at least once.

Q.12. When should you use Do Until loop?

The Do...Until loop should be used when the statement should be executed as long as the given condition is false. It can be written as a pretest or posttest loop. The pretest Do Until loop does not execute the statement if the condition is true from the beginning. The posttest loops executes the statement at least once.

Q.13. When should you use For...Next loop?

The For...Next loop should be used when the number of required iterations is known. It is a pretest loop that first initializes a counter variable to a starting value. It automatically increments the counter variable at the end of each iteration. The loop repeats as long as the counter variable is not greater than the end value.

Q.14. What Step value is used if it is not specified in For...Next loop?

The default Step value used in For...Next loop is 1.

Q.15. Which statements terminate Do and For loops early?

The Exit Do and Exit For statements are used to terminate loops before their natural termination.

Q.16. What should be Step value of For...Next loop if start value is greater than end value?
Logically, the Step value of For...Next loop should be negative if start value is greater than end value. Otherwise, the loop will become an infinite loop.

Q.17. What is nested loop?

A loop within a loop is called nested loop. In nested loops, the inner loop is executed completely with each change in the value of counter variable of outer loop.

Q.18. What is an infinite loop?

A loop in which the ending condition never occurs is called indefinite loop. It repeats forever until the user intervenes to stop the loop.

Q.19. Differentiate between counter and conditional loops?

Counter loop depends on the value of a variable known as counter variable. The value of counter variable is incremented or decremented each time the body of the loop is executed. This loop terminates when the value of counter variable reaches a particular value.

Conditional loop depends on special value known as sentinel value. Sentinel value indicates that the loop should continue or terminate. For example, a loop may execute while the value of a variable is not -1. Here -1 is the sentinel value used to terminate loop.

Q.20. Differentiate between outer and inner loop.

The loop that contains another loop in its body is called outer loop. The loop that is written in the body of an outer loop is called inner loop.

Q.21. How many times does the following loop execute?

a.

```
I = 10
Do While I > 1
I = I - 1
Loop
```

Answer: 9

b.

```
I = 10
Do While I >= 1
I = I - 1
Loop
```

Answer: 10

c.

```
I = 10
Do Until I > 1
I = I - 1
Loop
```

Answer: 0

d.

```
For I = 1 To 10
Next
```

Answer: 10

Q.22. Consider the following block of codes? What will be the output of both codes?

```
Dim Counter
Counter = 0
While Counter < 5
    Counter = Counter + 1
Print Counter
Wend
```

```
Dim Counter
Counter = 0
```



```
While Counter = 5
Counter = Counter + 1
Print counter
Wend
```

Answer: The preceding codes use While...Wend statement to increment a counter variable. In the first code, the statements in the loop are executed as long as the condition evaluates to True and prints 1, 2, 3, 4, 5. The second code does not print anything as the tested condition is not satisfied in the beginning of the While loop.

Multiple Choice

- The purpose of a repetition structure is to:
 - Execute a sequence of tasks once
 - Execute a procedure.
 - Repeat the execution of program statements
 - Repeat an If statement once.
- As a computer processes instructions, one execution of a loop is known as a(n):
 - Cycle
 - Duration
 - Iteration
 - Test
- A counter can be defined as:
 - The final value of a loop
 - A variable that counts the number of times a loop repeats
 - The initial value of a loop
 - The step value of a loop
- A loop within a loop is called:
 - Nested loop
 - Complex loop
 - Outer loop
 - Inner loop
- The loop which never ends is called:
 - Infinite loop
 - Running loop
 - Continuous loop
 - a and c
- The following is an example of:
 Do While Counter <= 500
 Loop
 - A pre-test loop that continues as long as the condition is false
 - A post-test loop that continues as long as the condition is true
 - A pre-test loop that continues as long as the condition is true.
 - A post-test loop that continues as long as the condition is false
- The following is an example of:
 Do
 <Steps to be repeated>
 Loop While Counter <= 500
 - A pre-test loop that continues as long as the condition is false.
 - A post-test loop that continues as long as the condition is false.
 - A pre-test loop that continues as long as the condition is true
 - A post-test loop that continues as long as the condition is true
- What is the value of j after the code in the loop below is completed?
 For j = 2 to 12 Step 3

10. A _____ value is used to indicate there is no more data to process.
11. A loop within a loop is called _____ loop.
12. The first loop encountered in a nested loop pair is called the _____ loop.

Answers

1. False	2. True	3. 1
4. 1,100,2	5. Do....Loop	6. Initial, final, step
7. Exit loop	8. Pre-test	9. Post-test
10. Sentinel	11. Nested loop	12. Outer

True / False

1. The repetition programming structure is used to make decisions in a program.
2. A repetition structure can be either a pretest loop or a posttest loop.
3. If a variable is incremented or decremented during each loop cycle and the loop condition depends on that variable, it is called a counter-controlled loop.
4. The condition in a pretest loop is evaluated before loop instructions are processed.
5. The condition in a posttest loop is evaluated after the loop instructions are processed.
6. In either pretest or posttest loops, the condition is evaluated with each repetition.
7. The instructions in a posttest loop are always processed at least once.
8. Pretest loops should be used only when you are certain that the loop instructions should be processed at least once.
9. The Do While...Loop is an example of pretest loop.
10. The condition used in Do While...Loop statement must evaluate to an integer value.
11. In Do Loop...While statement, the keyword While indicates that the loop instructions should be processed while the condition is false.
12. The Do Loop...While is an example of a pretest loop.
13. For...Next loops cannot be counted in reverse order.
14. For...Next loop exits when the loop variable exceeds the end value.
15. The initial value, end value, and Step value of counter variable in For loop must be numeric and can be either positive or negative.
16. The initial value, end value, and Step value of counter variable in the For loop control the number of times the loop body is processed.
17. For...Next loop can never result in an infinite loop.
18. Do Exit is used to terminate a loop.
19. Do...While Loop checks the condition before executing the statements in the loop.
20. The keyword that marks the end of a Do structure is End Do.
21. In a For ... Next loop, the counter is incremented at the beginning of the loop.
22. The statement that will immediately terminate a For...Next loop is End For.
23. A counter variable is normally incremented or decremented by 1.
24. The For...Next statement can be written within another For...Next statement.

Answers

1. F	2. T	3. T	4. T	5. T	6. T
7. T	8. F	9. T	10. F	11. F	12. F
13. F	14. T	15. T	16. T	17. F	18. F
19. T	20. F	21. F	22. F	23. T	24. T